

2110531 - Data Science and Data Engineering Tools

Spark on the Data Lakehouse: MetroRide

Asst. Prof. Natawut Nupairoj, Ph.D.

Department of Computer Engineering, Chulalongkorn University

natawut.n@chula.ac.th

CHULA ENGINEERING
Foundation toward Innovation

Why a table format on top of Parquet?

A folder of Parquet files

- Readers can see a half-finished write
- Two writers can overwrite each other
- Nothing enforces the schema
- Listing thousands of files is slow
- No history, no time travel

An open table format (OTF)

- Atomic commits: each one is a snapshot
- Concurrency control between writers
- Schema enforced, and can evolve
- File list and column stats in metadata → skip files
- Time travel and change feeds

Every lakehouse table has three layers



Engines

Compute: read, transform, write

Spark | DuckDB | Trino | Flink | pandas / Polars



Table format

Metadata: schema, snapshots, file lists

Apache Iceberg | Delta Lake | **DuckLake**



Storage

Data files

Parquet files on S3, ADLS, GCS or local disk

CHULA ENGINEERING
Foundation toward Innovation

How Spark reads a lakehouse table

1

Resolve

A catalog plugin looks up the table by name

2

Load snapshot

The current snapshot, or an older one for time travel

3

Plan files

List the snapshot's data files; skip files using partition values and min/max stats

4

Read in parallel

Executors read Parquet files, one task per file split

5

DataFrame

Continue with transformations and actions

driver: metadata only (small)

executors: data (large, parallel)

The table format decides which files belong to a snapshot. Spark never has to list the folder.

CHULA ENGINEERING
Foundation toward Innovation

How Spark writes: files first, commit last

1

Write files

Executors write new Parquet files; not visible yet

2

Collect

Driver gathers file paths, row counts and column stats

3

Commit

One atomic metadata change creates a new snapshot

4

Visible

New readers see the new snapshot; old snapshots stay readable



Job fails before commit

Readers never see partial data; orphan files are removed by maintenance later.



Two writers at once

Optimistic concurrency: the second commit detects the conflict and retries or fails.

DuckLake and Spark Integration

ducklake-spark connector

Spark DataSource V2 catalog from MotherDuck; executors write Parquet and the connector commits to DuckLake.

Pre-release jar · append-only · write-only (no reads)

JDBC + DuckDB driver

Spark sends queries to DuckDB, which attaches the DuckLake catalog.

Reads and writes work, but all data goes through DuckDB; writes can create many small files

ducklake-dataframe (PySpark)

Python reads the catalog; Spark reads and writes the Parquet files natively.

v1.0 proof of concept, not for production

Arrow bridge (this tutorial)

pyducklake reads and commits; Spark does all processing; Arrow carries the data.

Simple, no extra jars · data passes through the driver

Design: Spark computes, pyducklake commits

PySpark

- Read CSV, hash files
- Duplicate guards
- Parse, derive, validate
- Show: DataFrame API, Spark SQL

`append_from_spark()`

df.toArrow() → cast → append() in one transaction

`to_spark()`

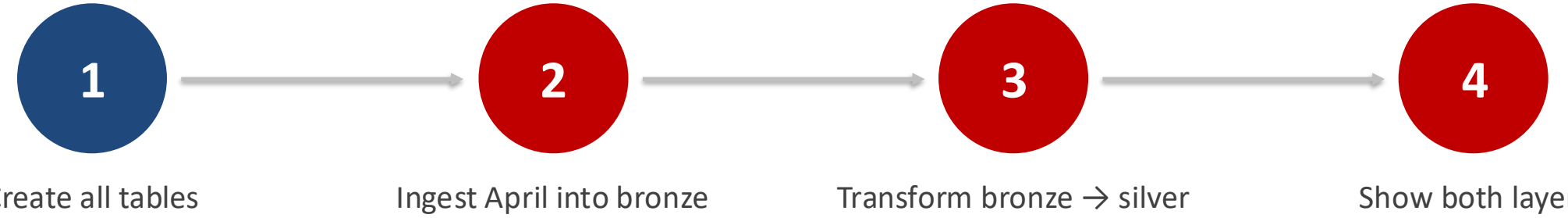
scan().to_arrow() → spark.createDataFrame()

pyducklake

- Create namespaces and tables
- Commit appends as snapshots
- Read tables as Arrow
- Snapshot history

With a native connector, executors would read and write Parquet directly and only the commit would touch the catalog. The ingest and transform code would not change.

Tutorial Steps





pyducklake: catalog work

PySpark: data work

CHULA ENGINEERING
Foundation toward Innovation

Setup · A local Spark session

```
spark = (
    SparkSession.builder
    .master("local[*]")
    .config("spark.sql.session.timeZone", "Asia/Bangkok")
    .config("spark.sql.execution.arrow.pyspark.enabled",
            "true")
    .config("spark.sql.shuffle.partitions", "4")
    .getOrCreate()
)
```

Requirements

```
$ java -version              # 17 or newer
$ uv add pyspark "pandas<3" # course uv project
# or: pip install pyspark "pandas<3"
```



`local[*]`

Driver and executors in one process, one task slot per CPU core.



`session.timeZone`

to_date() and hour() use it: Bangkok dates and time slots.



`arrow.pyspark.enabled`

Fast Spark ↔ Arrow ↔ pandas conversion.



`shuffle.partitions = 4`

Few partitions for small data instead of the default 200.

Setup · The Arrow bridge

```
def to_spark(table, snapshot_id=None):
    scan = table.scan()
    if snapshot_id is not None:
        scan = scan.with_snapshot(snapshot_id)
    return spark.createDataFrame(scan.to_arrow())

def append_from_spark(table_id, df, message):
    table = catalog.load_table(table_id)
    target = table.scan().with_limit(0).to_arrow().schema
    rows = df.select(*target.names).toArrow().cast(target)
    with catalog.begin_transaction() as txn:
        catalog.set_commit_message(
            message.format(n=rows.num_rows))
        txn.load_table(table_id).append(rows)
    return rows.num_rows
```



Read: scan → Arrow → Spark

Any table, at any snapshot, becomes a Spark DataFrame.



select(*target.names)

Columns by name: append() matches by position.



cast(target)

Exact table types: int32, decimal(10,2), timestamptz.



One transaction

One snapshot per call, with a commit message.

Step 1 · Create tables in pyducklake, read in Spark

```
bronze = catalog.create_table(
    ("bronze", "trips_raw"), Schema.of(
        *[optional(c, StringType())
          for c in TRIP_COLUMNS],
        required("source_file", StringType()),
        ...))

to_spark(silver).printSchema()
```

DuckLake type	Spark type
VARCHAR (StringType)	`string`
INTEGER (IntegerType)	`integer`
DECIMAL(10,2)	`decimal(10,2)`
TIMESTAMPTZ	`timestamp (shown in session time zone)`
DATE	`date`



Same schemas as the pyducklake tutorial: bronze keeps every source column as text plus source_file, source_row_number, file_sha256 and ingested_at; silver holds typed, validated trips with service_date and time_slot.

Step 2 · Ingest April with Spark

1 Fingerprint

binaryFile + sha2(content, 256)

G1 File guard

Hash already in bronze? Skip the file

2 Read as text

StringType schema; enforceSchema=False

3 Metadata

row_number() in file order; sha256; time

G2 Row guard

First copy per trip_id + left_anti join

4 Append

append_from_spark( ) with a commit message

132

April rows, snapshot 5

```
ingest_trips_csv(SOURCE_DIR / "trips_2026-04.csv")
# OK   trips_2026-04.csv: 132 rows read, 132 appended,
#      0 duplicates skipped
```

Step 2 · The Spark code behind the guards

```
# fingerprint the whole file
file_hash = (spark.read.format("binaryFile").load(path)
             .select(F.sha2("content", 256).alias("sha256"))
             .first()["sha256"])

# position in the file: 1, 2, 3, ...
file_order = Window.orderBy(F.monotonically_increasing_id())
batch = raw.withColumn("source_row_number",
                       F.row_number().over(file_order))

# first copy per trip_id, then drop trip_ids already in bronze
first_copy = Window.partitionBy("trip_id").orderBy("source_row_number")
new_rows = (batch
    .withColumn("copy_no", F.row_number().over(first_copy))
    .filter(F.col("copy_no") == 1).drop("copy_no")
    .join(bronze_now.select("trip_id"), "trip_id", "left_anti"))
```



sha2 on binaryFile

Same bytes, same hash: a replayed file is skipped whole.



Window + row_number

Ranks rows inside each trip_id; keep rank 1.



left_anti join

Keeps rows with no match on the other side.

Step 3 · Transform bronze to silver with Spark

1

Pick new rows

left_anti join of bronze against silver trip_ids

2

Parse and derive

try_to_timestamp, try_cast; to_date and hour in Bangkok time

3

Validate

Column rules; one row per trip_id with a window

```
typed = new_rows.select(
    F.try_to_timestamp(c("requested_at"), F.lit(TS_FORMAT))
        .alias("requested_at"),
    c("fare_thb").try_cast("decimal(10,2)").alias("fare_thb"),
    ...)
typed = (typed
    .withColumn("service_date", F.to_date("requested_at"))
    .withColumn("time_slot",
        F.when(F.hour("requested_at") == 8, "morning")
         .when(F.hour("requested_at") == 18, "evening")))
clean = typed.filter(valid_for_all
                     & (valid_completed | valid_cancelled))
```

132

to silver, 0 rejected, snapshot 6

ANSI mode: use try_ functions so a bad value becomes NULL instead of stopping the job.

Step 4 · Show both layers with Spark

```
bronze_df = to_spark(bronze)
silver_df = to_spark(silver)
bronze_df.createOrReplaceTempView("bronze_trips_raw")
silver_df.createOrReplaceTempView("silver_trips")
silver_df.select("trip_id", "fare_thb", ...).show(5)
```

```
spark.sql('''
  SELECT time_slot, count(*) AS requests,
         count_if(status = 'completed') AS completed,
         count_if(status = 'cancelled') AS cancelled,
         sum(fare_thb) AS gross_fare_thb
  FROM silver_trips GROUP BY time_slot''').show()
```

silver.trips after April

time_slot	requests	completed	cancelled	gross_fare_thb
morning	66	61	5	11,161.00
evening	66	63	3	11,414.00

Rows after April

132 bronze.trips_raw

132 silver.trips

CHULA ENGINEERING
Foundation toward Innovation

Step 5 · Append May without duplicate rows

+137

Nightly May load

New hash, new trip_ids: appended in snapshot 7

0

Replay of both files

File guard: sha2 already in bronze

0

Re-sent 3 May trips

Row guard: left_anti join finds every trip_id

bronze_rows	distinct_trip_ids	duplicate_rows
269	269	0

Checked with Spark: count vs. countDistinct("trip_id")

Steps 6–7 · Transform new rows, show both layers

+137

May rows into silver, snapshot 8 (127 completed, 10 cancelled)

0

rows on a second run: the transform is idempotent

Rows per month (Spark SQL)

month	bronze.trips_raw	silver.trips
2026-04	132	132
2026-05	137	137

May in silver, per time slot (DataFrame API)

time_slot	requests	completed	gross_fare_thb
morning	68	62	11,366.00
evening	69	65	11,723.00

Step 8 · Snapshot information

id	changes	commit_message
0–2	schemas_created: main, bronze, silver	
3–4	tables_created: bronze.trips_raw, silver.trips	
5	tables_inserted_into: [3]	bronze: ingest trips_2026-04.csv (pyspark)
6	tables_inserted_into: [4]	silver: transform 132 new trips (pyspark)
7	tables_inserted_into: [3]	bronze: ingest trips_2026-05.csv (pyspark)
8	tables_inserted_into: [4]	silver: transform 137 new trips (pyspark)

```
to_spark(silver, snapshot_id=6).count()   # 132  (after April)
to_spark(silver, snapshot_id=8).count()   # 269  (after May)
```

In changes, [3] and [4] are table ids: bronze.trips_raw and silver.trips.

Gotchas when Spark meets DuckLake



Gotcha	What to do
PySpark needs a JVM	Install Java 17 or newer; check java -version
PySpark warns about pandas 3	Pin pandas<3
Spark 4 ANSI mode: bad values raise errors	Use try_to_timestamp and try_cast
to_date() and hour() use the session time zone	Set spark.sql.session.timeZone
Every action re-runs the plan	cache() a DataFrame that feeds several actions
A window without partitionBy uses one partition	Fine for one small file; avoid on big data
toArrow() collects everything to the driver	Keep batches small, or use a native connector
pyducklake append() is positional	Select columns by name and cast to the table schema

Wrap-up

Key ideas

- Spark: the driver plans, executors process partitions; actions run jobs
- A table format turns Parquet files into tables with atomic snapshots
- Spark writes files first and commits metadata last
- DuckLake has no mature Spark connector yet; the tutorial bridges with Arrow

Try it yourself

1. Add one bad row to a copy of April; check it reaches bronze but not silver
2. Rewrite the Step 3 validation as one Spark SQL query on a temporary view
3. Build gold.daily_service_performance with Spark and commit it with append_from_spark()